HAMi 실험 보고서

측정 기간: 2026년 9월 2일 ~ 10월 14일 (7회)
측정 장비: RTX 4080 SUPER 1장, RTX 3060 1장, RTX 2060 SUPER 1장, RTX A6000 2장 (서버 4대)

1. 개요

1.1 검증한 가정

플랫폼은 물리 GPU 한 장을 여러 사용자에게 나눠 주는 방식을 택했습니다. 이 방식의 전제는 다음과 같습니다.

> 물리 GPU 한 장을 여러 개의 격리된 몫으로 나누어 각 몫에 VRAM 상한을 적용할 수 있고, 그 안에서 실제 딥러닝 작업이 정확하게 실행되며, 한 사용자가 다른 사용자의 작업을 망가뜨릴 수 없다.

이 전제의 네 요소(분할, 상한 적용, 정확성, 작업 격리)를 실험으로 확인했습니다. 실제 운영에 필요한 다섯 가지(몫 크기 계산, 다중 사용자 성능 분배, 두 장 카드, 여러 노드 운용, 재부팅과 전원 차단)도 함께 확인했습니다. 약한 호스트, 장시간 유지, 연산 제한 같은 그 밖의 결과는 4절에 정리했습니다. 분할에는 CUDA 호출을 사용자 공간에서 가로채는 HAMi-core를 사용했으며, Kubernetes 없이 Docker만으로 구성했습니다.

몫. 물리 GPU 한 장을 나눈 격리된 단위입니다. 사용자는 몫 하나를 받아 쓰며, 각 몫에는 VRAM 상한이 적용됩니다.

실행기. 이 보고서에서 실행기는 검증을 위해 직접 작성한 세션 관리 도구를 말합니다. 운영자 PC에서 동작하며, 사용자 요청을 받아 몫을 줄 노드와 장치를 정하고, 그 노드에서 컨테이너를 실행·감시·정리합니다. 5절 대책 중 "실행기가 ~합니다"라고 쓴 항목은 이 도구에 이미 넣었거나 앞으로 넣어야 할 기능입니다. 구성은 4.9절에 정리했습니다.

1.2 결론 요약

항목	결론
분할·상한 적용·정확성·작업 격리	**성립합니다.** 설정한 상한이 그대로 적용되었고, 실제 작업 5종이 분할 상태에서도 비트 단위로 같은 결과를 냈습니다. 이웃 작업이 다른 사용자의 결과를 바꾼 경우는 없었습니다
몫 크기 계산	몫 하나에 드는 고정 비용이 카드마다 다릅니다. 이 비용은 **SM 수 × SM당 최대 스레드**로 예측할 수 있고 VRAM 용량과는 관계가 없습니다. 첫 실험에서 얻은 일괄 규칙(설정값 − 256)은 카드마다 맞지 않으므로 **노드를 등록할 때 보정이 필요합니다**
다중 사용자	같은 작업이라면 한 장을 9명까지 나눠도 성능이 1/N로 고르게 나뉘고, 카드 전체 처리량은 97~99 %로 유지됩니다. 수용 인원은 VRAM 용량이 결정합니다. 성격이 다른 작업을 섞으면 남는 GPU 시간이 큰 작업에 더 돌아가면서도, 작은 작업의 지연은 거의 늘지 않았습니다
두 장 카드	상한과 VRAM 차감이 장치마다 따로 적용됩니다. 두 장에 나눠 돌리는 분산 학습(DDP, PyTorch DistributedDataParallel)도 결과가 정확하므로 두 장 신청을 허용할 수 있습니다
여러 노드	실행기가 서로 다른 GPU의 두 노드를 하나의 풀로 운용하면서 과할당이나 회수 누락 없이 동작했습니다

항목	결론
재부팅과 전원 차단	실제 전원 손실 뒤 전원을 다시 켜자, 노드가 사람 개입 없이 정상 상태로 돌아왔습니다. 실행 중이던 세션은 다시 시작되지 않으며, 체크포인트는 저장이 끝난 마지막 시점까지 복구되었습니다
할당된 몫 바깥의 VRAM 점유 차단	**성립하지 않는 경로가 있습니다.** 사용자 코드 한 줄로 몫 바깥의 물리 메모리를 사실상 제한 없이 점유할 수 있고, 그 양은 스레드가 많은 카드일수록 커집니다(A6000에서 2g 몫이 약 10GB 점유). 악의 없는 평범한 사용에서도 같은 일이 생길 수 있습니다(예: 직접 작성한 커널의 로컬 메모리, 미검증). 운영 전에 호스트 쪽 대책이 필요합니다

확인된 결함과 대책은 5절에 정리했습니다.

2. 실험 환경과 통제 변수

2.1 장비

항목	RTX 4080 SUPER	RTX 3060	RTX 2060 SUPER	RTX A6000 × 2
아키텍처 / CC	Ada / 8.9	Ampere / 8.6	Turing / 7.5	Ampere / 8.6
SM × SM당 최대 스레드	80 × 1536	28 × 1536	34 × 1024	84 × 1536
VRAM	16GB	12GB	8GB	48GB
CPU / RAM	i7-9700KF 8코어 / 31GB	i7-11700 / 15GB	Pentium G4400 2코어 / 7GB	Xeon Gold 6226R 16코어 / 125GB
비고	단독 서버	디스플레이 연결	약한 호스트	ECC 꺼짐, NVLink 없음, 다른 연구실 소유

2.2 모든 실험에서 통제한 변수

구분	고정한 값	이유
드라이버	580.178.04 (CUDA 13.0), 모든 장비 동일	컨텍스트 크기와 예약분이 드라이버에 따라 달라짐
HAMi-core	커밋 `f01e9f23` 고정, 상위 저장소의 빌드 절차를 그대로 사용. 3060, 2060 SUPER, A6000 서버의 `libvgpu.so` 는 비트 단위로 같은 파일(sha256 `d75afe70…`)	릴리스 태그가 없어 커밋으로 고정
컨테이너 런타임	Docker + NVIDIA Container Toolkit 1.20.0, `nvidia` 런타임은 등록하지 않고 툴킷 훅만으로 `--gpus` 사용	장비 간 조건 통일
작업 이미지	`pytorch/pytorch:2.13.0-cuda13.0-cudnn9-runtime` 기반 (PyTorch 2.13.0+cu130, cuDNN 9.20), 장비마다 같은 베이스 digest와 같은 패키지 버전	워밍업 뒤 손실과 초과분이 이미지에 따라 달라짐
컨테이너 사양	`--network none` , 비루트 uid, 작업 디렉터리와 `libvgpu.so` 는 읽기 전용 마운트, `--shm-size` 2~4g	외부 영향 차단, 운영 조건 재현
비교 조건	라이브러리 없음(네이티브) / 라이브러리만 주입 / 상한까지 적용, 세 조건을 같은 작업으로 비교	가로채기 비용과 상한의 효과를 분리

구분	고정한 값	이유
측정 기준	메모리는 **호스트 NVML v2**만 사용(컨테이너 안 NVML은 HAMi가 가상화하므로 근거로 쓰지 않음). 0.1초(일부 0.5초) 간격으로 표본을 얻고, 프로세스는 cgroup 경로로 컨테이너와 대응	드라이버 예약분이 섞인 v1 값과 가상화된 값을 배제
측정 도구 검증	정확히 2,048 MiB를 잡는 프로브로, 표본기가 2,048.0 MiB 차이를 보고하는지 먼저 확인	도구 오차 배제
정확성 판정	학습 종료 시점 모델 파라미터 전체의 SHA-256. 난수 시드, 결정론적 알고리즘, 데이터 로더 워커의 시드와 수를 고정. 아키텍처가 다른 카드끼리는 비교하지 않고 **같은 카드의 단독 실행**과 비교	손실값의 근사 일치가 아니라 비트 일치로 판정
표기	메모리 단위는 MiB·GiB로 적고, 몫 크기의 `2g` 는 2 GiB를 뜻함	단위 혼동 방지
할당 단위	사용 가능량은 OOM이 날 때까지 32 MiB 단위로 할당해 측정(첫 장비는 256 MiB 단위)	할당 단위에서 오는 양자화 오차 제거
외부 부하	대상 GPU에 다른 프로세스가 없고 사용량이 유휴값과 같을 때만 측정 시작. 측정 중 다른 프로세스가 나타나면 즉시 중단(공유 장비)	다른 사용자의 사용량이 섞이는 것을 배제
데이터	모든 데이터와 가중치를 미리 받아 두고, 측정 중에는 네트워크 없이 실행. 장비 간 데이터 파일 md5 일치	다운로드 속도와 데이터 차이 배제

2.3 실험별로 바꾼 변수

실험	독립 변수	추가로 고정한 변수	측정값
상한 적용	설정값 2·4·8·16 GiB	카드, 이미지	사용 가능량, 호스트 물리 정점
분할 상태 정확성	작업 5종, 몫 크기(최소 통과 몫, 정점 이하)	시드, 배치, epoch	SHA, 처리량
카드별 상수	카드 4종, 워밍업 유무	몫 크기와 무관함을 2g·4g(·8g·16g)로 확인	컨텍스트 차감량, 손실, 초과분
차감량의 요인	스레드당 스택 한도 1→16 KiB	빈 컨텍스트, 같은 카드	물리 사용량 기울기
다중 사용자	동시 사용자 수 2~10, 작업 혼합	같은 몫 크기(1,728 MiB), 같은 작업	개별 지연, 합산 처리량, SHA
약한 호스트	사용자 수, 로더 워커 4·1·0	같은 몫(1,856 MiB), 컨테이너별 RAM 상한, swap 차단	처리량, 작업당 RAM, CPU
작업 격리	이웃의 동작 4종(OOM 반복, 긴 커널, 강제 종료, 라이브러리 없이 전체 점유), 옆 장치 포화	대상 작업과 몫	대상 작업의 SHA와 지연
두 장 카드	장치별 상한 조합, 채우는 순서, NCCL P2P 사용 여부, HAMi 유무	같은 모델·데이터·합 배치	장치별 사용 가능량, SHA, 처리량
몫 바깥 점유	스택 한도 4·16·32·64·120 KiB, 모듈 로딩 방식	몫 2g	호스트 물리 정점, HAMi가 본 사용량
연산 제한	`CUDA_DEVICE_SM_LIMIT` 100~10 %	작업 종류	상대 처리량
장시간·운용	시간(3·12시간), 노드 연결 끊김, 실행기 교체, 동시 요청 폭주	몫 크기, 작업	상한, 회계, SHA, 용량 복귀

실험	독립 변수	추가로 고정한 변수	측정값
재부팅·전원 차단	정상 재부팅, 전원 차단, 체크포인트 저장 방식	몫 2g, 작업	복구 상태, 세션 분류, 체크포인트 복구, 용량 복귀

3. 실험 결과 — 분할, 상한 적용, 정확성, 작업 격리

3.1 VRAM 상한 적용

- 설정값을 넘는 할당은 파이썬에서 처리할 수 있는 OOM 예외로 거절되며, 프로세스는 종료되지 않습니다. 예외가 난 뒤에도 계산을 이어 가고, 메모리를 반환한 뒤 다시 할당할 수 있었습니다. 75초 동안 상한을 1,096회 연달아 넘겨도 드라이버 오류(Xid)는 한 건도 없었습니다.
- 상한은 **프로세스가 아니라 몫 단위로** 적용됩니다. 한 컨테이너의 자식 프로세스 셋이 각각 1,536 MiB를 요구하면 세 번째 요청이 거절됩니다.
- NCCL 통신 버퍼, 사용자가 컴파일한 CUDA 확장의 드라이버 직접 호출, cudart를 정적 링크한 실행 파일도 모두 상한 안에서 집계되었습니다.

3.2 분할 상태의 정확성

작업	데이터	최소 통과 몫	정확성	처리량 (네이티브 → 주입 → 분할)
ResNet-18	CIFAR-10	1,728 MiB	비트 일치	3,853 → 3,821 → 3,825 img/s
ResNet-50	Imagenette	6,720	비트 일치	330.8 → 331.5 → 330.5 img/s
BERT-base	SST-2	3,712	비트 일치	490.8 → 489.3 → 488.8 ex/s
GPT-2	WikiText-2	6,784	혼란도 일치	19,580 → 19,555 → 19,521 tok/s
Stable Diffusion 1.5	—	4,608	이미지 해시 일치	1.200 → 1.194 → 1.195 img/s

20회 실행 결과가 모두 분할하지 않은 기준과 같았습니다. 가로채기 비용은 처리량 기준 1 % 미만이었습니다.

3.3 이웃 작업의 영향

이웃의 동작	대상 작업 결과	대상 작업 지연
몫 안에서 OOM 반복 (75초간 1,096회)	비트 일치	1.04배
3초짜리 커널 연속 실행	비트 일치	1.95배
실행 중 강제 종료	비트 일치	1.31배
라이브러리 없이 카드 전체 점유	비트 일치	1.04배
같은 서버의 옆 장치를 OOM까지 채우고 유지 (A6000)	비트 일치	1.004배

이웃 작업 때문에 속도가 느려질 수는 있지만 결과는 바뀌지 않았습니다. 강제 종료된 컨테이너의 메모리는 0.22초 만에 반환되었습니다.

4. 실험 결과 — 운영에 필요한 수치

4.1 몫 하나에 드는 고정 비용

몫 하나의 메모리 양은 세 가지 값으로 나타냅니다. **설정값**은 운영자가 정하는 상한, **사용 가능량**은 사용자가 실제로 쓸 수 있는 양, **물리 사용량**은 카드에서 실제로 소모되는 양입니다. 설정값과 나머지 두 값의 차이는 몫 크기와 관계없는 고정 비용이었습니다(2g~16g에서 모두 같음).

항목 (MiB)	2060 SUPER	3060	4080 SUPER	A6000
드라이버 예약	407	387	455	600
풀 (유휴 여유량)	7,784	11,891	15,904	48,538
컨텍스트 차감량 (HAMi가 몫에서 미리 빼는 양)	90	104	250	262
빈 컨텍스트 1개의 실제 물리 크기 (유휴 카드, 프로세스 1개)	93	112	290	271
손실, 워밍업 없음 (설정값 − 사용 가능량)	96	128	—	288
손실, cuBLAS·cuDNN 워밍업 후	160	192	256	**352**
초과분 (물리 정점 − 설정값)	+14	0	+32	+26

컨텍스트 차감량과 빈 컨텍스트 크기는 **카드를 몇 명이 나눠 쓰는지와 관계없는, 컨텍스트 하나당 값**입니다. 다른 프로세스가 없는 유휴 카드에서 프로세스 하나가 빈 컨텍스트를 만들 때 늘어난 장치 사용량을 빈 컨텍스트 크기로, 같은 프로세스가 몫 안에서 본 여유량의 감소를 차감량으로 측정했습니다. 컨텍스트는 GPU를 쓰는 프로세스마다 하나씩 생깁니다. 따라서 카드를 N명이 나눠 쓰면 이 비용도 N번 들고, 한 사용자가 프로세스를 두 개 띄우면 자기 몫에서 두 번 빠집니다. 3060의 112 MiB에는 디스플레이 연결로 인한 8 MiB가 포함되어 있습니다.

손실과 초과분도 몫 크기(2g~16g)와 관계없이 같았으므로, 몫 하나(컨테이너 안 프로세스 하나)당 고정 비용으로 볼 수 있습니다.

A6000 두 장은 모든 값이 MiB 단위까지 같았습니다. 같은 모델의 카드라면 장치마다 다시 측정할 필요가 없습니다.

손실 중 작업 공간 부분은 할당 API를 거치므로 HAMi가 몫 안에서 제한합니다. 반면 초과분은 HAMi가 보지 못하는 컨텍스트 증가분(커널 코드 적재 등)입니다. 따라서 위 초과분은 측정한 작업에서의 값이며, 실행하는 커널에 따라 달라질 수 있습니다(5.1절).

4.2 컨텍스트 차감량의 예측식

HAMi-core 소스를 확인한 결과, 컨텍스트 차감량은 고정 상수가 아니라 **초기화 시점에 그 카드에서 측정한 빈 컨텍스트의 크기**입니다.

이 크기를 좌우하는 것은 스택 예약입니다. 여기서 **스택**은 CPU 프로그램의 호출 스택과 같은 개념으로, GPU 커널의 각 스레드가 함수 호출, 지역 변수, 레지스터에 다 들어가지 않는 값을 두는 스레드 전용 메모리(CUDA 용어로 로컬 메모리)입니다. **스택 한도**는 스레드 하나가 쓸 수 있는 스택 크기의 상한이며 기본값은 1 KiB입니다. 드라이버는 어떤 스레드가 실제로 스택을 쓸지 모르므로, 컨텍스트를 만들 때 **카드에 동시에 올라갈 수 있는 모든 스레드(SM 수 × SM당 최대 스레드)** 몫의 스택을 VRAM에 미리 잡아 둡니다.

빈 컨텍스트에서 스택 한도를 늘리면 물리 사용량이 정확히 `SM 수 × SM당 최대 스레드 ÷ 1024` MiB/KiB씩 늘어났습니다(3060 42.0, 2060 SUPER 34.0, A6000 126.0). 이 보고서에서는 이 값을 **스택 기울기**(스택 한도를 1 KiB

늘릴 때 늘어나는 물리 사용량)라고 부릅니다. 컨텍스트 크기가 카드마다 달라지는 주된 이유는 모든 하드웨어 스레드에 대한 스택 예약입니다.

> **컨텍스트 차감량 ≈ 26 MiB + 1.87 KiB × (SM 수 × SM당 최대 스레드)**

이 식은 세 카드(2060 SUPER 34,816 스레드 → 90 MiB, 3060 43,008 → 104 MiB, 4080 SUPER 122,880 → 250 MiB)의 차감량 실측값을 **카드의 총 스레드 수(SM 수 × SM당 최대 스레드)를 변수로 한 선형 회귀**로 맞춘 결과입니다. 기울기 1.87 KiB는 스레드 하나당 드는 비용입니다. 그중 1 KiB는 기본 스택 한도로 설명되고, 나머지 약 0.87 KiB는 드라이버의 다른 스레드별 예약으로 보입니다. 절편 26 MiB는 스레드 수와 관계없는 고정 비용입니다. 기울기의 두 부분(1 KiB와 0.87 KiB)을 더 쪼개어 확인하지는 않았습니다.

측정점이 세 개뿐이므로, 식을 세울 때 쓰지 않은 A6000으로 검증했습니다. **측정 전에 기록한 예측**은 261.6 MiB였고 실측은 262 MiB(오차 0.1 %)였습니다. A6000은 4080 SUPER와 SM 구성이 거의 같고(84 대 80 SM, 둘 다 SM당 1536) VRAM은 세 배입니다. 그런데도 두 카드의 차감량 차이(12 MiB)가 스레드 수 차이로 예측한 값(11.2 MiB)과 일치했습니다. 즉 **VRAM 용량은 차감량에 영향을 주지 않습니다.**

워밍업 뒤 손실은 차감량에 cuBLAS·cuDNN 작업 공간 약 70~90 MiB를 더한 값이었습니다.

4.3 일괄 규칙 "설정값 − 256"의 한계

첫 실험(4080 SUPER)에서 얻은 규칙 "사용 가능량 = 설정값 − 256"은 다른 크기의 카드에서는 실제 값과 차이가 납니다.

- **작은 카드에서는 지나치게 보수적입니다.** 2060 SUPER에서는 몫마다 96 MiB가 낭비되며, 8명이면 768 MiB입니다.
- **큰 카드에서는 실제보다 많이 안내합니다.** A6000의 손실은 352 MiB이므로, 이 규칙을 따르면 몫마다 96 MiB를 더 약속하게 됩니다. 사용자는 안내받은 양을 다 쓰기 전에 OOM을 겪습니다. 이 결과는 측정 전에 기록한 예측(320~350 MiB로 256보다 큼)과 같은 방향입니다.

4.4 등급표

등급은 인원 N에 대해 `N × (등급 + 64) ≤ 풀`을 만족하는 가장 큰 64 MiB 정렬 값입니다. 사용자가 실제로 쓸 수 있는 양은 등급에서 4.1절의 워밍업 뒤 손실을 뺀 값입니다.

카드당 인원	2060 SUPER	3060	4080 SUPER	A6000
1	7,680	11,776	15,808	48,448
2	3,776	5,824	7,872	24,192
3	2,496	3,840	5,184	16,064
4	1,856	2,880	3,904	12,032
6	1,216	1,856	2,560	8,000
8	896	1,408	1,920	5,952

4080 SUPER에서는 각 등급을 해당 작업으로 가득 채워 확인했습니다(8인: ResNet-18 실습, 4인: BERT-base 미세 조정, 3인: Stable Diffusion 추론, 2인: ResNet-50·GPT-2). 등급을 2의 거듭제곱으로 잡으면 같은 카드에서 한 자리가 줄어듭니다(2 GiB면 7명, 1,920 MiB면 8명).

4.5 사용자 수에 따른 성능 분배

같은 작업(ResNet-18)을 여러 명이 동시에 실행한 결과입니다. 지연은 단독 실행 대비 배수입니다.

동시 사용자	개별 지연 (평균)	카드 전체 처리량	최악/최선 편차	정확성
2	2.00	0.971	1.002	전부 일치
4	3.90	0.978	1.016	전부 일치
6	5.78	0.985	1.028	전부 일치
8	7.67	0.993	1.040	전부 일치
9	8.62	0.991	1.049	전부 일치
10	9명 정상, 10번째는 시작 시 OOM으로 정상 거절	—	—	9명 일치

- 지연은 인원에 비례했고(오차 4 % 이내), 카드 전체 처리량은 거의 줄지 않았습니다.
- **수용 인원의 한계는 VRAM에서 왔습니다.** 10번째 사용자는 이미 실행 중인 9명에게 영향을 주지 않고 시작 단계에서 거절되었습니다.
- 8명이 동시에 시작해도 초기화 실패가 없었고, 13초 안에 모두 학습에 들어갔습니다.

성격이 다른 작업을 섞은 경우. 큰 작업 1개와 작은 작업 여러 개를 함께 실행했습니다.

함께 실행한 작업	인원	큰 작업의 지연	작은 작업의 지연
ResNet-50 1개 + ResNet-18 5개	6	**3.31배**	5.85~5.97배
BERT 1개 + ResNet-18 6개	7	6.39배	6.41~6.65배

ResNet-50은 1/6보다 많은, 약 1/3의 GPU 시간을 썼습니다. 그러나 작은 작업이 그만큼 손해를 본 것은 아닙니다. 작은 작업의 지연(5.85~5.97배)은 같은 작업 6개를 함께 돌렸을 때(5.78배)보다 1~3 % 늘어나는 데 그쳤습니다. 즉 **큰 작업은 작은 작업이 쓰지 않고 남긴 GPU 시간을 가져갔다**고 보는 것이 맞습니다. BERT와 섞은 경우에도 작은 작업의 지연은 인원수(7배)보다 작았습니다.

원인(추정). 연산 제한을 걸지 않으면 HAMi는 연산을 나누지 않으며, 여러 컨테이너의 프로세스는 GPU 드라이버의 기본 시분할을 그대로 공유합니다. 드라이버는 프로세스별 작업 대기열을 번갈아 실행하고, 차례가 온 프로세스에 대기 중인 커널이 없으면 바로 다음 프로세스로 넘어갑니다.

- ResNet-50처럼 큰 커널을 미리 여러 개 넣어 두는 작업은 차례가 올 때마다 그 시간을 끝까지 씁니다.
- ResNet-18처럼 작은 커널을 짧게 실행하고 CPU 쪽 작업(커널 실행 요청, 동기화, 데이터 준비)을 기다리는 작업은 차례가 와도 실행할 커널이 없는 경우가 많아 차례를 일찍 넘깁니다.

작은 작업이 넘긴 시간을 큰 작업이 쓰므로, 카드는 쉬지 않고 일하고 작은 작업은 원래 필요한 만큼의 시간을 받습니다. 드라이버의 시분할 단위와 전환 규칙은 공개되어 있지 않아, 이 설명은 측정 결과에서 추론한 것입니다.

측정한 작업에서는 작은 작업이 인원수보다 더 느려진 사례가 없었습니다. 다만 GPU에 짧은 연산을 넣고 결과를 기다리기를 자주 반복하는 작업(예: batch 1 추론)은 연산을 넣을 때마다 이웃의 차례가 끝나기를 기다려야 하므로, 인원수보다 더 느려질 가능성이 있습니다. 이 경우는 아직 측정하지 않았습니다(7절).

4.6 약한 호스트

2코어 Pentium과 7GB RAM을 갖춘 2060 SUPER 서버에서 ResNet-18(몫 1,856 MiB)을 1~4명이, i7 서버의 3060에서는 1~6명이 실행했습니다(총 56건).

- 56건 모두 정상 완료되었고 해시가 일치했습니다. 호스트 OOM, swap, 재시도는 없었습니다.
- 2코어 호스트는 단독 실행부터 CPU 사용률이 100 %였지만, 처리량(1,794 img/s)은 i7 호스트(1,821 img/s)와 거의 같았습니다. 로더 워커를 0개로 줄여도 같았습니다. 다만 두 서버는 GPU도 달라(2060 SUPER, 3060), 호스트 성능만 비교한 결과는 아닙니다.
- 작업당 RAM은 약 1.0~1.2GB였습니다. RAM 기준 수용 인원(5명)보다 VRAM 등급 기준(4명)이 작아, **VRAM이 먼저 한계가 되었습니다.**
- 4080 SUPER 서버(8코어)에서는 8명이 각자 로더 워커 4개를 쓰면 CPU가 포화해 시작이 느려졌습니다.
- JPEG 디코딩처럼 CPU 전처리가 많은 작업은 측정하지 않았습니다(7절).

4.7 두 장 카드

한 사용자에게 A6000 두 장을 함께 줄 수 있는지 확인한 실험입니다. 세 가지를 확인했습니다.

1. 한 컨테이너에 두 장을 넣었을 때 상한과 VRAM 차감이 장치마다 따로 적용되는가
2. 두 장에 나눠 학습해도 결과가 정확한가
3. 장치 간 통신이 몫 바깥에 숨은 메모리를 만들지 않는가

세 가지 모두 문제가 없었으므로 두 장 신청을 허용할 수 있습니다. 단, 이 서버에서 NCCL은 공유 메모리(SHM)로만 통신했으므로 3번은 SHM 경로에 대한 결론이며, 장치 간 직접 전송(P2P) 경로는 확인하지 못했습니다(7절). 다만 상한을 지정하는 방식에 실수하기 쉬운 점이 있고(5.6절), HAMi 초기화가 장치 0을 건드리는 현상은 아직 확인 중입니다(5.7절).

확인 항목	결과
장치별 상한의 독립성	한 컨테이너에 장치 0 = 4g, 장치 1 = 8g를 걸고 한쪽을 OOM까지 채워도 다른 쪽의 여유량은 변하지 않았습니다. 채우는 순서를 바꿔도 같았고, 물리 정점은 각 상한 이내였습니다
컨텍스트 차감	각 장치에서 따로 262 MiB씩 차감되었습니다
상한 지정 방식	장치별 값(`CUDA_DEVICE_MEMORY_LIMIT_<i>`)이 공통값보다 우선합니다. **장치별 값만 주고 공통값을 주지 않으면, 지정하지 않은 장치에는 상한이 없습니다.** `<i>` 는 호스트 번호가 아니라 **컨테이너 안 순번**입니다
장치 간 복사	속도와 체크섬이 네이티브와 같았고, 대상 버퍼도 몫의 사용량에 정상 반영되었습니다. 숨은 메모리는 없었습니다
두 장 분산 학습(DDP)	ResNet-18을 두 프로세스(rank)로 나눠 학습했습니다. 다섯 구성(네이티브, HAMi, HAMi 반복, P2P 끔 ×2)의 결과 SHA가 **모두 같았고**, HAMi의 처리량 손실은 1.3~1.7 %였습니다
NCCL 통신 버퍼	한 장에서도 두 장에서도 몫 안에서 집계되었습니다. 4 GiB 몫을 3.5 GiB까지 채운 뒤 NCCL을 초기화하면 정확히 4 GiB에서 거절되었고, 상한이 없는 대조군은 5GB까지 늘었습니다
통신 경로 (A6000 서버)	P2P를 끄지 않아도 NCCL은 공유 메모리(SHM)로 통신했습니다. 장치 간 복사 속도도 PCIe 직접 전송 기대치보다 낮았습니다

4.8 장시간 유지와 반복 실행

- **12시간 유지:** 4 GiB 몫을 12시간 유지하며 10분마다 학습을 실행했습니다. 상한(3,456 MiB), 새 프로세스가 본 여유량(3,469 MiB), 학습 결과 해시가 끝까지 변하지 않았습니다. 73회 작업이 모두 완료되었고 Xid는 없었으며, 종료 후 카드 사용량은 시작 전과 같았습니다.
- **3시간 반복 실행:** 스크립트로 작업을 3시간 동안 쉬지 않고 시작·종료하며 204건을 처리했습니다. 모두 성공했고, 매 회차 용량이 원래 값으로 돌아왔으며, 남은 프로세스나 새어 나간 용량은 없었습니다.

4.9 여러 노드 운용

실행기로 세대가 다른 두 노드(3060, 2060 SUPER)를 하나의 풀로 운용했습니다. 실행기는 노드에 상주 프로세스나 설치를 남기지 않으며, 연결할 때마다 SSH로 에이전트를 보내 실행합니다.

구성 요소	역할
에이전트	노드에서 컨테이너를 실행·조회·삭제하고, 종료를 감지하며, 0.5초마다 호스트와 컨테이너별 GPU 사용량을 수집
용량 계산	노드의 장치별로 설정값 + 64 MiB를 차감하고 과할당하지 않음
배치	가장 알맞은 장치에 배정(best-fit). 특정 노드에만 들어가는 작업이 대기 중이면 그 노드를 예약해 뒤의 작은 작업이 먼저 들어가지 못하게 함
시작 조절	노드별 시작 간격 1초, 시작 후 30초 안의 OOM은 1회 재시도(5.5절)
종료 분류	정상, 몫 부족(CUDA OOM), 호스트 메모리 부족, 시간 초과, 취소, 노드 재시작, 연결 유실 등으로 구분
상태 재구성	연결할 때마다 노드에 남은 컨테이너로부터 실행 상태를 다시 만듦

시험	결과
동시 요청 폭주 (합산 풀의 약 3배, 24건)	모두 정상 처리. 풀 초과 0회(50 ms 간격 감시), 물리 사용량이 설정값 + 64를 넘은 경우 0건, 용량 정상 복귀
실행 중 SSH 연결 강제 끊김 (3회)	1~2초 안에 재연결. 이중 계산이 없었고, 세션 6개 모두 할당을 빠짐없이 수행
실행기 비정상 종료 후 새 실행기 인수	남은 컨테이너 4개를 모두 찾아 차감했고, 풀 초과 없음
특정 노드에만 들어가는 큰 작업	그 노드를 예약하지 않으면 뒤의 작은 작업에 밀려 계속 대기. 노드 예약 규칙으로 해결
원격 제어 지연	중계 서버를 거치는 요청 하나에 약 8 ms. 시작 지연은 노드의 Docker가 좌우(`docker run` 중앙값 0.23~0.32초)
시작 경합 재현 시도	같은 노드에 50쌍(컨테이너 100개)을 동시에 시작했으나 재현되지 않음

4.10 사용자 작업 환경

- 권한 없는 사용자(읽기 전용 이미지, uid 고정, 모든 권한 제거)로도 사용자 영역 패키지 설치, Jupyter Lab, TensorBoard, 몫 한도까지의 GPU 할당이 모두 동작했습니다.
- 단, **홈을 Docker 기본 임시 파일 시스템에 두면** 실행 권한이 없어 Jupyter(pyzmq)와 TensorBoard(grpc)가 원인을 알기 어려운 오류로 종료됩니다. 실행 가능한 볼륨을 홈으로 주면 정상 동작합니다.
- 컴파일러를 포함한 개발용 이미지에서 권한 없는 사용자가 CUDA 커널을 컴파일해 22초 만에 실행했고, 상한과 우회 차단은 그대로 유지되었습니다. 이미지 크기는 6.3GB에서 21.3GB로 늘어납니다.

4.11 연산 제한

핵심: 연산 제한은 제한한 작업의 GPU 사용량을 낮출 뿐, 이웃이 GPU를 다 쓰지 못하는 상황에서는 이웃 작업을 빠르게 하지 않습니다. 남는 시간은 이웃이 GPU를 기다리고 있을 때만 이웃에게 돌아가고, 그렇지 않으면 쓰이지 않습니다. 따라서 측정한 범위에서 연산 제한은 **경합할 때 우선순위를 주는 효과**만 있습니다.

동작 방식. HAMi의 연산 제한(`CUDA_DEVICE_SM_LIMIT`)은 GPU 스케줄러에 개입하지 않고, **각 프로세스가 자기 사용률을 보고 커널 실행을 스스로 늦추는** 방식입니다.

1. 컨테이너 안에 장치별 토큰 통이 있고, 감시 스레드가 주기적으로 그 프로세스의 GPU 사용률을 읽습니다.
2. 사용률이 설정값보다 낮으면 토큰을 많이, 높으면 적게 채웁니다.
3. 프로세스가 커널을 실행하려 하면 HAMi가 그 호출을 가로채 커널 크기만큼 토큰을 뺍니다. 토큰이 모자라면 잠시 대기(`nanosleep`)한 뒤 다시 확인합니다.
4. 일단 실행된 커널에는 개입하지 않습니다.

이 판단에는 "지금 GPU를 원하는 다른 작업이 있는가"라는 정보가 들어가지 않습니다. 드라이버의 시분할은 일이 없으면 차례를 바로 넘기므로 빈 시간이 생기지 않지만, 연산 제한은 다른 작업이 없어도 토큰이 찰 때까지 기다립니다. 그동안 GPU는 아무도 쓰지 않습니다.

측정 결과 (상대 처리량, 제한 없음 = 1.00)

설정	큰 행렬곱 연속	작은 커널 다수	ResNet-18 학습
100 %	1.00	1.00	1.00
75 %	0.97	0.89	—
50 %	0.92	0.96	0.54
25 %	0.74	0.54	0.25
10 %	**0.32**	0.21	—

- **긴 커널은 설정값을 크게 넘습니다.** 연산 제한은 커널 실행 직전에만 개입하므로, 한번 실행된 긴 커널은 끝까지 실행됩니다. 10 %로 설정해도 큰 행렬곱의 처리량은 제한이 없을 때의 32 %였습니다. 8인 등급(12.5 %)이라면 큰 커널을 쓰는 작업이 설정값의 2~3배를 쓸 수 있습니다. ResNet-18처럼 작은 커널을 쓰는 작업은 거의 설정값대로 줄었습니다.
- **남는 시간은 버려집니다.** 제한 없는 작업과 30 %로 제한한 작업을 함께 실행했을 때, 제한 없는 작업이 먼저 끝나 카드가 비었는데도 제한된 작업은 계속 30 % 근처로 대기했습니다.
- **경합 시에는 우선순위로 동작합니다.** 두 작업이 모두 GPU를 계속 쓰는 상황에서 한쪽에만 제한을 걸면, 다른 쪽이 확실히 앞섰습니다. 소유 연구실의 작업을 외부 사용자의 작업보다 앞세우는 용도로 쓸 수 있습니다.
- **혼합 작업에서는 이득이 없습니다.** 4.5절의 혼합 작업에서도 작은 작업은 지연이 거의 늘지 않았습니다. 이 상황에서 큰 작업을 제한하면 작은 작업은 빨라지지 않고 카드 전체 처리량만 줄어듭니다.

따라서 연산 제한은 공정성을 위한 기본 설정으로 쓰지 않고, 경합 시 우선순위를 정하는 용도로만 씁니다. 또한 설정값이 실제 사용률의 상한을 보장하지 않으므로, 이 값을 성능의 백분율로 안내해서는 안 됩니다.

4.12 노드 재부팅과 전원 차단

운영 중인 노드는 예고 없이 재부팅되거나 꺼질 수 있으며, 소유자가 따로 있는 장비는 더욱 그렇습니다. 3060과 2060 SUPER 두 노드에서 정상 재부팅(9월 30일)과 실제 전원 차단(10월 14일)을 시험했습니다.

정상 재부팅. 실행기로 세션 3개를 실행한 상태에서 두 노드를 재부팅했습니다. 실행기는 boot_id 변화를 감지해 세 세션을 모두 노드 재부팅(`node_reboot` , exit 137)으로 분류했고, 풀 용량은 원래 값으로 돌아왔습니다. 상한 초과와 재시도는 없었습니다.

전원 차단. 각 노드에서 2g HAMi 세션 3개와 호스트 파일 쓰기 프로세스를 실행한 상태로 현장에서 전원 코드를 뽑았습니다. 세션은 실행기 없이 Docker로 직접 실행했습니다. 두 노드는 약 26분 뒤 전원 버튼으로 다시 켜졌습니다. 세션과 호스트 파일은 다음 저장 방식을 나눠 썼습니다.

방식	동작
`none`	체크포인트를 저장하지 않음 (세션만 해당)
`atomic`	임시 파일에 쓰고 fsync → rename → 디렉터리 fsync
`plain`	기존 파일을 그 자리에서 덮어씀
`rename_nofsync`	임시 파일에 쓴 뒤 fsync 없이 rename (호스트 파일만 해당)

확인 항목	결과
전원 손실 여부	두 노드 모두 직전 부팅에 정상 종료 기록이 없고 boot_id가 바뀌어, 실제 전원 손실로 확인했습니다
시스템 복구	전원을 다시 켠 뒤에는 사람 개입 없이 정상 상태로 돌아왔습니다. 파일 시스템 오류와 dpkg 중단 흔적이 없었고, 드라이버가 정상 적재되었으며(Xid 없음) Docker가 자동으로 기동했습니다. 부팅에는 31.9초(3060 노드)와 53.9초(2060 SUPER 노드)가 걸렸습니다
GPU와 HAMi	유휴 사용량이 차단 전 값(9 / 1 MiB)으로 돌아왔고, HAMi 공유 파일은 남지 않았습니다. 2g 몫의 상한, OOM 뒤 연산, 종료 후 사용량이 차단 전과 같았습니다
세션 컨테이너	모두 exit 255로 남았고 자동으로 다시 시작되지 않았습니다. Docker가 기록한 종료 시각(`FinishedAt`)은 차단 시각이 아니라 부팅 뒤 Docker가 기동한 시각이었습니다
체크포인트	`atomic` 과 `plain` 모두 저장이 끝난 마지막 체크포인트를 정상적으로 불러왔습니다. 잃은 진행분은 4~17 step(약 3~10초)이었고, `none` 세션은 진행분 전체를 잃었습니다
호스트 파일	세 방식 모두 마지막으로 끝난 쓰기까지 해시가 맞았습니다
커널 변경	차단 당일 자동 업데이트(unattended-upgrades)가 설치한 새 커널로 부팅했습니다. 3060 노드에서는 차단 6분 전에 설치가 끝났습니다. 드라이버는 그대로여서 HAMi 동작과 유휴값은 변하지 않았습니다

해석과 한계.

- 노드가 정전되어도 노드 디스크의 체크포인트는 저장이 끝난 시점까지 복구할 수 있었습니다. 다만 시행은 노드 두 대에서 한 번씩뿐입니다.
- `plain` 방식은 저장 도중 전원이 끊기면 하나뿐인 체크포인트가 깨질 수 있습니다. 이번에 깨지지 않은 것은 차단 순간이 저장 중이 아니었기 때문일 수 있습니다.
- 노드 디스크의 체크포인트는 백업이 아닙니다. 디스크 고장, 노드 회수, 재설치 때는 함께 사라지므로, 노드 밖 저장소에 사본이 있어야 합니다(7절의 저장소 경로).

- 현장에서 전원 버튼으로 켰기 때문에, 전원 코드를 꽂는 것만으로 노드가 켜지는지(BIOS AC 복구 설정)는 확인하지 못했습니다.

운영에 반영할 사항

결과	대책
정전 뒤 Docker의 종료 시각이 차단 시각과 다름	실행기는 boot_id 변화로 노드 재부팅을 분류하고, 종료 시각은 마지막 heartbeat나 감시 기록으로 정합니다. 정전 상황에서 실행기의 분류는 아직 확인하지 않았습니다(7절)
마지막 체크포인트 이후의 진행분은 잃음	사용자에게 체크포인트를 주기적으로 저장하고 **최소 두 개를 남겨 두도록** 안내합니다(새 이름으로 저장한 뒤 오래된 것을 지우고, 최신 파일이 열리지 않으면 이전 파일로 이어서 학습). Hugging Face `save_total_limit` 처럼 설정 하나로 지원되며(Lightning `save_top_k` 는 지표 기준이므로 `monitor="step", mode="max"` 를 함께 지정), 디스크 사용량이 약 2배가 되므로 저장 공간 할당량에 반영합니다. 이 방식 자체는 시험하지 않았습니다. 실행기가 체크포인트 도우미를 제공한다면 그 안에서는 원자적 저장을 씁니다
자동 업데이트가 커널을 설치함	운영 노드는 업데이트 시간대를 정하거나 커널을 고정합니다. 보정은 카드 모델, 드라이버, 작업 이미지가 바뀔 때만 다시 하고, 커널만 바뀌었으면 2g 상한·OOM 뒤 연산·유휴값을 확인하는 짧은 시험으로 대신합니다
정전 뒤 자동 기동 여부를 모름	무인 운영 전에 BIOS AC 복구 설정을 확인합니다. 이 설정이 켜진 노드는 전기가 들어오면 이미 켜진 상태이므로, 이때 전원 버튼을 누르면 Ubuntu 기본 설정에 따라 정상 종료가 시작됩니다. 현장 복구 절차에도 반영합니다

5. 확인된 결함과 대책

5.1 할당된 몫 바깥의 VRAM 점유 (가장 중요한 결함)

HAMi는 컨텍스트를 초기화 시점의 크기로 한 번만 몫에서 빼고, 그 뒤 **컨텍스트 자체가 커지는 것은 보지 못합니다.**
사용자가 직접 일으킬 수 있는 두 경로를 2g 몫에서 확인했습니다.

- **스택 한도 증가:** 사용자는 `cudaDeviceSetLimit` 한 줄로 스레드당 스택 한도(4.2절)를 바꿀 수 있습니다. 드라이버는 늘어난 크기 × 카드의 모든 스레드만큼 VRAM을 추가로 잡는데, 이 증가는 할당 API를 거치지 않아 HAMi가 보지 못합니다.
- **즉시 모듈 로딩:** 사용자는 환경 변수(`CUDA_MODULE_LOADING=EAGER`)로 커널 코드를 처음부터 모두 적재하게 할 수 있습니다. 이 적재도 할당 API를 거치지 않아 HAMi가 보지 못합니다.

두 경우 모두 할당 API로 상한을 넘는 요청은 계속 거절되었지만, 물리 점유는 막을 수 없었습니다.

경로	사용자가 할 일	2060 SUPER	A6000
기준	없음	2,028	2,074
스택 한도 4 KiB	`cudaDeviceSetLimit(cudaLimitStackSize, 4K)` 한 줄	—	2,450 (×1.2)
스택 한도 16 KiB	같음	—	3,962 (×1.9)
스택 한도 32 KiB	같음	—	5,978 (×2.9)
스택 한도 64 KiB	같음	4,168 (×2.0)	**10,010 (×4.9)**
스택 한도 120 KiB	같음	**6,108 (×3.0)**	약 17,000 (×8.3, 식으로 계산)
즉시 모듈 로딩	`CUDA_MODULE_LOADING=EAGER`	2,564 (+516)	3,018 (+944)

(단위: 호스트에서 측정한 컨테이너 물리 정점, MiB. 모든 경우 HAMi가 본 몫 안 사용량은 변하지 않았습니다.)

- 물리 정점은 `설정값 + 초과분 + 스택 기울기 × (스택 한도 − 기본값 1 KiB)` 로 계산되며, A6000의 네 측정값은 이 식과 2 MiB 이내로 맞았습니다. 스택 기울기는 4.2절의 `SM 수 × SM당 최대 스레드 ÷ 1024` MiB/KiB(A6000 126, 2060 SUPER 34)입니다. 예를 들어 A6000에서 64 KiB면 2,048 + 26 + 126 × 63 ≈ 10,012 MiB입니다. **스레드가 많은 큰 카드일수록 같은 코드 한 줄의 피해가 커집니다.**
- 물리 메모리가 부족해지면 **다음에 들어온 사용자가 하드웨어 OOM으로 실패합니다.** 다른 사용자의 결과가 틀려지지는 않으므로 정확성과 작업 격리는 유지되지만, 할당된 몫 바깥의 VRAM 점유는 막지 못합니다.
- 정상 작업에서도 HAMi가 보지 못하는 증가분이 수십 MiB 있습니다(A6000 약 52 MiB). 측정한 작업에서는 이 증가분이 초과분과 여유분 64 MiB 안에 들어갔습니다.

HAMi가 보는 메모리와 보지 못하는 메모리. 라이브러리와 사용자 코드가 쓰는 GPU 메모리는 두 경로로 잡힙니다.

경로	예	HAMi	결과
할당 API(`cuMemAlloc` , `cuMemCreate` 등)	텐서, cuBLAS·cuDNN 작업 공간, NCCL 통신 버퍼	호출마다 계산하고 몫을 넘으면 거절	몫 안에서 동적으로 제한됨 (손실에 포함)
드라이버가 컨텍스트 안에서 직접 잡는 메모리	커널 코드 적재, 스레드별 로컬 메모리(스택) 예약, printf 버퍼, 디바이스 힙	가로채지 않음	몫 바깥 물리 점유 (초과분)

두 번째 경로의 크기는 **실행하는 커널에 따라 달라지므로 미리 알 수 없습니다.** 기본값인 지연 로딩에서는 실제로 쓰는 커널만 올라가므로, 일반적인 PyTorch 작업에서는 수십 MiB에 그쳤습니다. 다만 이는 측정한 작업 범위에서 얻은 경험치이며, Triton, FlashAttention, 직접 컴파일한 확장처럼 커널을 많이 올리는 작업은 측정하지 않았습니다.

사용자가 직접 작성한 커널 (미측정, 추정). 스택 한도 경로는 `cudaDeviceSetLimit` 을 명시적으로 호출하지 않아도 생길 수 있습니다. 커널이 스레드마다 큰 지역 배열을 쓰거나 레지스터가 모자라 값이 로컬 메모리로 넘치면(spill), 드라이버는 그 커널을 실행할 때 `스레드당 로컬 크기 × 카드의 모든 스레드 슬롯` 만큼 예약을 **스스로 늘립니다.** 예를 들어 스레드당 16 KiB를 쓰는 커널은 A6000에서 약 2GB(16 × 126 MiB)를 몫 바깥에 잡을 것으로 예상됩니다. 이 경우에는 가로챌 API 호출이 없고, **악의가 없는 일반 사용자**(CUDA 실습생, 개발용 이미지에서 확장을 컴파일하는 연구자)에게도 생길 수 있습니다. CUDA의 동작 방식에서 추론한 내용이며, 2g 몫에서 지역 배열 크기를 바꿔 가며 확인하는 실험은 아직 하지 않았습니다(7절).

대책

원인이 여러 갈래이고 일부는 가로챌 호출이 없습니다. 따라서 **원인과 관계없이 실제 물리 사용량을 보는 호스트 쪽 장치를 기본 방어선**으로 두고, HAMi-core 수정은 보조 수단으로 씁니다.

우선순위	대책	내용	상태
1	호스트 감시 후 종료	실행기가 컨테이너별 물리 정점이 설정값 + 여유분을 넘으면 종료합니다. 모든 경로에 동작하지만, 감시 간격 사이에 늘어나는 양은 막지 못합니다. 큰 카드는 짧은 시간에 많은 메모리를 잡으므로, 감시 간격(0.1초)과 종료 지연을 측정해 피해 상한을 정해야 합니다	미검증 (현재는 초과 표시만 함)
1	실측값 기반 배정	배정 차감을 `max(설정값 + 여유분, 실측 물리 사용량)` 으로 계산합니다. 이미 몫을 넘긴 컨테이너가 있는 장치에는 새 사용자를 넣지 않으므로, 새 사용자가 하드웨어 OOM을 겪지 않습니다. 실행 중 증가는 막지 못합니다	미검증

우선순위	대책	내용	상태
2	HAMi-core 수정	`cuCtxSetLimit` 호출을 가로채 증가분(Δ × SM × 스레드)을 몫의 사용량에 더하고, 몫이 부족하면 거절합니다. 커널 실행 중 자동으로 늘어나는 경우는 막지 못합니다. 커널 실행을 가로채 커널별 로컬 크기를 조회할 수는 있지만 모든 실행에 비용이 붙습니다. 모듈 적재량은 미리 알기 어렵습니다	미검증
3	실행기에서 `CUDA_MODULE_LOADING=LAZY` 지정	즉시 모듈 로딩 경로를 예방합니다	사용자가 프로세스 안에서 바꿀 수 있어 단독으로는 불충분
—	사용자 안내	사용자가 커널을 직접 작성하는 환경(CUDA 실습, 개발용 이미지)에서는 정상적으로 사용해도 몫을 넘을 수 있음을 안내합니다	제안
—	표기 수정	대책이 검증될 때까지 구현 계획의 "VRAM 용량 격리 보장"을 "할당 API 기준의 상한 보장"으로 낮춰 표기합니다	제안

풀의 여유분을 늘리는 방법은 해결책이 아닙니다. 스택 경로의 증가폭에는 사실상 제한이 없기 때문입니다.

5.2 일괄 규칙으로 계산한 몫 크기의 오차

4.3절처럼 일괄 규칙으로 계산하면 작은 카드에서는 몫을 낭비하고, 큰 카드에서는 실제보다 많은 양을 안내합니다.

대책: 노드를 등록할 때 보정을 필수 단계로 둡니다. 보정 도구는 작성해 두었으며, 카드 한 장에 약 30초가 걸립니다.

1. 유휴 상태의 NVML 값(총량, 예약, 여유)을 읽어 풀로 정합니다.
2. 상한을 건 프로브로 컨텍스트 차감량, 워밍업 뒤 사용 가능량, 호스트 물리 정점을 측정합니다.
3. 아래 식으로 배정량과 등급을 계산하고, **카드 모델 + 드라이버 + 작업 이미지** 단위로 저장합니다. 드라이버나 이미지를 바꾸면 다시 보정합니다.

> 배정 차감(debit) = 설정값 + 여유분, 여유분 = max(64, ceil32(초과분 + 32)), ceil32는 32 MiB 단위 올림
> 사용자에게 표시할 사용 가능량 = 설정값 − (워밍업 뒤 손실)

보정을 생략해야 한다면, 4.2절의 예측식에 작업 공간 90 MiB를 더해 "설정값 − (차감량 + 90)"으로 안내하는 방법이 차선책입니다. 네 카드에서 실측과 맞았습니다.

여유분은 보정 작업(행렬곱, 합성곱, 학습 몇 스텝)과 실제 작업 5종에서 얻은 **경험치이며 보장값이 아닙니다.** 초과분은 실행하는 커널에 따라 달라지므로(5.1절), 여유분을 넘는 경우는 5.1절의 호스트 감시와 실측값 기반 배정으로 대응합니다.

사용자에게는 설정값이 아니라 사용 가능량을 보여 줘야 합니다. 배정량과 실제 사용 가능량이 다르면 혼란이 생깁니다.

5.3 환경변수 주입 시 상한 우회

라이브러리를 `LD_PRELOAD` 환경변수로만 주입하면, 사용자가 변수를 지우는 것만으로 상한이 사라졌습니다(4 GiB 몫에서 6 GiB 할당 성공).

대책: `/etc/ld.so.preload`에 읽기 전용으로 주입합니다. 이 방식에서는 변수를 지워도, 로더를 복사해 써도, 정적 링크 바이너리를 써도 상한이 유지되었습니다(검증됨). 사용자가 이 파일을 고칠 수 있으면 상한이 무의미해지므로, **사용자에게 관리자 권한을 줄 수 없습니다.** 시스템 패키지가 필요한 경우 관리자에게 요청해 이미지에 반영하는 절차를 마련해야 합니다. 또한 이 파일이 있는 컨테이너는 GPU 라이브러리 없이 프로세스를 시작할 수 없으므로, CPU 전용 작업용 이미지를 따로 둡니다.

5.4 비동기 할당자에서 CUDA 그래프 캡처 실패

PyTorch의 `cudaMallocAsync` 백엔드를 쓰면, HAMi가 스트림 캡처 중 장치 번호를 잘못 계산해 CUDA 그래프 캡처가 실패했습니다.

대책: 이미지에서 할당자 설정을 기본값으로 고정해, 사용자가 비동기 백엔드를 켜지 못하게 합니다. 기본 할당자에서는 CUDA 그래프와 컴파일 최적화가 모두 정상 동작했고 상한도 지켜졌습니다(검증됨).

5.5 동시 시작 경합

두 컨테이너가 같은 순간에 시작하고 한쪽이 곧바로 큰 메모리를 잡았을 때, 다른 쪽이 컨텍스트를 만들지 못하고 종료한 사례가 한 번 있었습니다. 당시 기록에는 이웃의 사용량이 일시적으로 자기 사용량처럼 집계되어 있었습니다. 이후 같은 크기 조건을 포함해 50쌍을 동시에 시작했지만 재현되지 않았습니다. 이 환경에서는 컨테이너 안 NVML이 같은 PID 네임스페이스의 프로세스만 보여 주므로, 이론상으로는 다른 컨테이너의 사용량을 잘못 읽을 수 없습니다.

대책: 실행기가 노드별로 시작 간격을 1초 두고, 시작 후 30초 안에 생긴 OOM은 한 번 재시도합니다. 8명 기준 비용은 약 8초입니다. 효과가 입증되지는 않았지만, 비용이 거의 없는 예방책이므로 유지합니다.

5.6 장치별 상한 지정의 함정 (두 장 카드)

HAMi에서 VRAM 상한은 환경변수로 지정하며, 방법은 두 가지입니다.

- `CUDA_DEVICE_MEMORY_LIMIT=8g` : 컨테이너에 보이는 **모든 장치**에 같은 상한(공통값)
- `CUDA_DEVICE_MEMORY_LIMIT_1=8g` : **1번 장치에만** 상한(장치별 값, 공통값보다 우선)

문제는 장치별 값만 주고 공통값을 주지 않을 때 생깁니다. A6000 두 장이 보이는 컨테이너에 `CUDA_DEVICE_MEMORY_LIMIT_1=8g`만 주면, 1번 장치는 8 GiB로 제한되지만 **0번 장치에는 상한이 전혀 없어** 48 GiB 전체를 쓸 수 있었습니다(4.7절). 이 문제는 컨테이너에 장치가 둘 이상 보일 때 생깁니다. 예를 들어 두 장을 신청한 사용자에게 장치마다 다른 크기의 몫을 주면서 한 장치의 값을 빠뜨린 경우, 또는 실행기가 `--gpus all`로 모든 장치를 넣고 상한만으로 사용 범위를 정하려는 경우입니다. 상한을 주지 않은 장치는 그 사용자가 제한 없이 쓸 수 있습니다.

또한 `<i>`는 호스트의 GPU 번호가 아니라 **컨테이너 안에서 보이는 순번**입니다. `--gpus device=1`로 호스트 1번만 넣으면 컨테이너 안에서는 그 장치가 0번이 되므로, `_0`으로 지정해야 합니다.

대책: 실행기는 컨테이너에 보이는 **모든 장치에 상한을 명시**합니다(공통값을 함께 주거나 장치마다 `_<i>`를 지정). `<i>`는 컨테이너 안 순번으로 계산합니다. 한 장만 배정하는 컨테이너는 `--gpus device=<n>`으로 그 장치만 보이게 합니다.

5.7 HAMi 초기화가 장치 0을 건드리는 현상 (미확인)

HAMi를 켠 두 장 분산 학습에서, 두 번째 프로세스(rank1)가 자기 장치뿐 아니라 **장치 0에도 최대 210 MiB의 물리 사용량**을 남겼습니다. 실행 런처도 장치 0에 10~64 MiB를 남겼습니다(네이티브에서는 없음). 소스상 HAMi는 초기화할

때 장치 0에서 컨텍스트 크기를 측정한 뒤 해제하므로, 그 흔적으로 보입니다. 일시적인지 지속적인지는 아직 구분하지 못했습니다.

대책(잠정): 장치를 하나만 보이게 한 컨테이너에서는 그 장치가 컨테이너 안에서 0번이 되므로 영향이 없었습니다. 두 장 이상을 보이게 하는 컨테이너는, 장치 0이 가득 찬 상태에서도 시작되는지 확인하기 전까지 같은 서버의 장치들을 함께 비워 배정합니다.

5.8 그 밖의 운영 요건

결과	대책
작업이 끝날 무렵 큰 메모리를 요구하면(평가 배치를 학습 배치보다 크게 잡은 GPT-2) 전체의 94 % 지점에서 실패	평가 배치를 학습 배치보다 크게 잡지 않도록 안내합니다. 실행기가 몫의 90 %·97 %에서 경고를 남기고 남은 시간을 추정합니다
실패 원인(호스트 메모리 부족, 외부 종료, 몫 부족)을 호스트에서 구분해 관찰할 수 있음	세 가지를 구분해 안내합니다. 몫 부족은 "배정받은 GPU 메모리가 부족합니다"로 표시합니다
작은 등급에서는 컨텍스트 하나가 몫에서 큰 비중을 차지해(4080 SUPER에서 250 MiB 이상), 8인 등급에서 노트북을 두 개 열면 두 번째가 시작되지 않음	실습 등급을 한 단계 올리거나, 세션당 Jupyter 커널을 하나로 제한하고 이를 명시합니다
성격이 다른 작업을 섞으면 큰 작업이 1/N보다 많은 GPU 시간을 쓰지만, 큰 작업이 가져간 것은 작은 작업이 쓰지 않은 시간이므로 작은 작업의 지연은 거의 늘지 않음. 연산 제한은 사용량을 낮출 뿐 이웃을 빠르게 하지 않음(4.5·4.11절)	연산 제한을 기본으로 걸지 않습니다. 소유 연구실 작업을 앞세우는 등 경합 시 우선순위가 필요한 경우에만 적용하고, 설정값을 성능 백분율로 표시하지 않습니다. 짧은 연산을 넣고 결과를 자주 기다리는 작업이 손해를 보는지(7절) 확인한 뒤 다시 판단합니다
컨테이너 안 NVML은 가상화된 값을 보여 줌	사용량 집계와 과금은 호스트 쪽 측정값으로 합니다
홈이 실행 불가 파일 시스템이면 패키지가 동작하지 않음	세션마다 실행 가능한 볼륨을 홈으로 줍니다
실행기를 교체하면 이전 실행기의 종료된 컨테이너가 남음	새 실행기가 남은 컨테이너를 인수해 분류하고 삭제합니다
네트워크를 끈 컨테이너에서 `torchrun --standalone`이 컨테이너 이름을 찾지 못해 멈춤	두 장 작업은 `127.0.0.1` 고정 주소로 랑데부합니다
노드마다 작업 이미지를 따로 빌드하면 이미지 ID가 달라지고, 버전을 고정하지 않은 패키지가 빌드 시점에 따라 바뀜	레지스트리의 같은 digest를 배포하고, Dockerfile의 패키지 버전을 모두 고정합니다
호스트 한 대의 수용 인원	`N_max = min(VRAM 등급 인원, floor((유휴 가용 RAM − 0.5GB) ÷ 1.2GB))` 로 정합니다. 카드가 두 장인 서버에서는 이 식을 확인하지 않았습니다. CPU 전처리가 많은 작업은 따로 측정합니다

6. 플랫폼 설계에 반영할 사항

1. **풀의 용량은 GPU(장치) 단위로 관리합니다.** 상한과 VRAM 차감이 GPU별로 독립적이므로, 노드가 아니라 GPU마다 남은 용량을 계산합니다. 같은 모델의 카드는 보정값을 공유합니다.
2. **두 장 신청을 허용할 수 있습니다.** 한 컨테이너에서 두 장으로 분산 학습해도 결과가 정확하고 처리량 손실이 작습니다. 같은 노드의 두 장치에 각각 몫을 걸고, 5.6·5.7절의 조건을 지킵니다. 통신 경로(P2P 또는 SHM)는 서버마다 다를 수 있습니다.
3. **할당된 몫 바깥의 VRAM 점유는 대책을 검증하기 전까지 막을 수 없다고 표기합니다.** 특히 A6000처럼 큰 카드를 여러 명이 나눠 쓰면, 한 사람이 여러 사람의 작업을 실패시킬 수 있습니다.

4. **소유자가 따로 있는 장비는 사전 협의가 필요합니다.** 설치·설정 변경 범위와 반납 시 되돌리기 절차, 소유 연구실이 GPU를 쓸 때 그만큼을 풀에서 빼는 방법, 측정·점유 시간대를 미리 정해야 합니다. 장비가 예고 없이 빠질 수 있으므로 사용자 데이터는 노드 바깥에 둡니다.
5. **구형 카드는 별도 이미지가 필요합니다.** CUDA 13은 Maxwell·Pascal·Volta를 지원하지 않아, 검증한 이미지는 지원 범위상 GTX 10 시리즈에서 실행되지 않을 것입니다(직접 시험하지 않음). Turing 이전 카드를 받으려면 CUDA 12 이미지를 따로 유지해야 하므로, 장비 조사 전에 수용 기준을 정해야 합니다.

7. 미검증 항목

항목	확인할 내용
몫 바깥 점유 대책	호스트 감시 종료의 지연과 그 사이 최대 점유량, 실측값 기반 배정, HAMi-core 패치의 차단 효과와 정상 작업에 주는 영향
커널 로컬 메모리 자동 확장	2g 몫에서 스레드당 지역 배열 크기를 바꿔 가며 커널을 실행해, `cudaDeviceSetLimit` 없이도 몫 바깥 예약이 늘어나는지와 그 크기
작업 종류별 초과분	Triton, FlashAttention, JAX, 직접 컴파일한 확장 등 커널을 많이 올리는 작업에서 여유분 64 MiB가 지켜지는지
작은 작업이 인원수보다 더 느려지는 경우	GPU에 짧은 연산을 넣고 결과를 기다리기를 자주 반복하는 작업(예: batch 1 추론)을 긴 연산을 쉬지 않고 넣는 작업과 함께 실행해, 같은 작업끼리 나눠 쓸 때보다 지연이 뚜렷이 늘어나는지. 늘어난다면 연산 제한으로 줄어드는지
HAMi 초기화의 장치 0 접촉	장치 0이 가득 찬 상태에서 두 장 컨테이너가 시작되는지, 남는 메모리가 지속되는지
세 노드 이상의 풀	A6000(장치 두 개)을 포함한 이기종 배치, 중간 서버를 거치는 두 단계 접속, 경로 장애, 외부 사용량 반영
저장소 경로	NAS 마운트(SMB만 가능, NFS 꺼짐), 비루트 권한, 체크포인트, NAS 장애 시 컨테이너 동작
노드 재시작과 정전	Docker 데몬 재시작 시 상태 복구(공유 서버라 운영자 승인 필요), 실행기로 띄운 세션의 정전 뒤 분류, BIOS AC 복구 설정(정전 뒤 자동 기동), 체크포인트 두 개 유지 방식의 효과
장치 간 직접 전송	A6000 서버에서 NCCL이 P2P를 쓰지 않고 복사 속도가 낮은 원인
CPU 부하가 큰 작업	JPEG 디코딩이 많은 작업에서 약한 호스트의 수용 인원
최소 등급 실측	작은 카드의 최소 등급(896 / 1,408 MiB)에 실제로 들어가는 작업

별첨 1. 측정 일정

기간	장비	다룬 주제
9월 2일	RTX 4080 SUPER	상한 적용, 정확성, 작업 격리, 실패 형태, 연산 제한, 3시간 반복 실행
9월 2일	RTX 4080 SUPER	다중 사용자, 동시 시작, 등급표, 권한 없는 사용자, 단계별 메모리
9월 3일	RTX 4080 SUPER	NCCL 통신 버퍼, 12시간 유지, 개발용 이미지, 시작 경합 발견
9월 23일 ~ 28일	RTX 3060, RTX 2060 SUPER	카드별 상수와 예측식, 약한 호스트, 두 노드 풀 운용, 몫 바깥 점유 발견
9월 30일	RTX 3060, RTX 2060 SUPER	정상 재부팅
10월 6일 ~ 7일	RTX A6000 × 2	예측식 검증, 큰 카드의 몫 바깥 점유, 두 장 카드

기간	장비	다룬 주제
10월 14일	RTX 3060, RTX 2060 SUPER	전원 차단

모든 측정 기간에 Xid는 한 건도 기록되지 않았습니다.

별첨 2. HAMi-core 환경변수 (소스에서 추출)

변수	의미
`CUDA_DEVICE_MEMORY_LIMIT` / `_<i>`	장치별 메모리 상한(g/m/k 접미사 또는 바이트). `_<i>`가 우선하고, 둘 다 없으면 상한 없음. `<i>`는 컨테이너 안 순번
`CUDA_DEVICE_SM_LIMIT` / `_<i>`	연산 사용률 백분율, 기본 100
`CUDA_DEVICE_MEMORY_SHARED_CACHE`	회계용 공유 메모리 파일, 기본 `/tmp/cudevshr.cache` (컨테이너마다 `/tmp`가 따로 있으므로 묶 단위로 동작)
`LIBCUDA_LOG_LEVEL`	로그 수준 0–4
`GPU_CORE_UTILIZATION_POLICY`	연산 제한기 강제 적용·해제
`ACTIVE_OOM_KILLER`	값을 읽기는 하지만 이 커밋에서는 사용하지 않음

메모리 제한은 `cuMemAlloc_v2`, `cuMemAllocManaged`, `cuMemAllocPitch_v2`, `cuMemAllocAsync`, `cuMemCreate`를 가로채, 추적 중인 사용량과 요청량의 합을 상한과 비교하는 방식입니다. `cuCtxSetLimit`은 가로채지 않습니다(5.1절의 원인).

별첨 3. 검증 작업

작업	데이터	구성	기준 결과
ResNet-18	CIFAR-10	3 epoch, batch 128	acc 0.825, 3,853 img/s (4080 SUPER)
ResNet-50	Imagenette-320	3 epoch, batch 64, 224px	acc 0.499, 331 img/s
BERT-base	GLUE SST-2	1 epoch, batch 32, seq 64	acc 0.906, 491 ex/s
GPT-2	WikiText-2	1 epoch, batch 4, 512 블록	PPL 49.88 → 21.95
Stable Diffusion 1.5	—	512×512, 25 step, fp16	1.20 img/s
ResNet-18 (두 장)	합성 데이터 32×32	합 배치 128, 100 step	고정 시드, rank별 SHA 비교